

Adding Audio for Processing 4 on Apple Silicon Macs

A Conversation with Gemini

i am working with Processing IDE on Apple Silicon.

I have a Python sketch that is trying to use the Sound library and getting this error:

```
java.lang.ClassCastException: class
jdk.internal.loader.ClassLoaders$AppClassLoader cannot be cast to class
java.net.URLClassLoader (jdk.internal.loader.ClassLoaders$AppClassLoader
and java.net.URLClassLoader are in module java.base of loader 'bootstrap')
at jycessing.LibraryImporter.addJarToClassLoader(LibraryImporter.java:315)
at
jycessing.LibraryImporter.recursivelyAddJarsToClasspath(LibraryImporter.java:16
4)
at jycessing.LibraryImporter.addLibrary(LibraryImporter.java:140)
at jycessing.LibraryImporter$1. __call__ (LibraryImporter.java:82)
at org.python.core.PyObject. __call__ (PyObject.java:480)
at org.python.core.PyObject. __call__ (PyObject.java:484)
at org.python.pycode. _pyx18.f$0(PythonAudioPlaySampleWav.pyde:1)
at org.python.pycode. _pyx18.call_function(PythonAudioPlaySampleWav.pyde)
at org.python.core.PyTableCode.call(PyTableCode.java:171)
at org.python.core.PyCode.call(PyCode.java:18)
at org.python.core.Py.runCode(Py.java:1614)
at org.python.core.Py.exec(Py.java:1658)
at
org.python.pycode. _pyx17.f$0(/Users/george/Documents/GitHub/PythonAudioPl
aySampleWav/PythonAudioPlaySampleWav.pyde:96)
```

```
at
org.python.pycode._pyx17.call_function(/Users/george/Documents/GitHub/PythonAudioPlaySampleWav/PythonAudioPlaySampleWav.pyde)
at org.python.core.PyTableCode.call(PyTableCode.java:171)
at org.python.core.PyCode.call(PyCode.java:18)
at org.python.core.Py.runCode(Py.java:1614)
at org.python.core.Py.exec(Py.java:1658)
at org.python.util.PythonInterpreter.exec(PythonInterpreter.java:276)
at jycessing.PAppletJythonDriver.processSketch(PAppletJythonDriver.java:233)
at
jycessing.PAppletJythonDriver.findSketchMethods(PAppletJythonDriver.java:613)
at jycessing.Runner.runSketchBlocking(Runner.java:399)
at jycessing.mode.run.SketchRunner.lambda$2(SketchRunner.java:112)
at java.base/java.lang.Thread.run(Thread.java:833)
Hmm. Can't load native code to bring window to front using the absolute path:
/Users/george/Documents/Processing/modes/PythonMode/mode/libjniOSX.dylib.
Hmm. Can't load native code to bring window to front.
This is a known, deeply frustrating compatibility issue.
The core of the problem is that Processing Python Mode (Processing.py) is
fundamentally broken in Processing 4+.
```

Why is this happening?

1. **The ClassLoader Error:** The `java.lang.ClassCastException` happens because Python Mode was built for older versions of Java (Java 8). Processing 4 uses a much newer version of Java, which changed how class loading works. When Python Mode's `add_library()` function tries to forcefully inject the Sound library's `.jar` files into the Java classpath, Java denies it, throwing this crash.
2. **The Apple Silicon (`.dylib`) Error:** The second warning at the bottom indicates that Python Mode's built-in window management utilities (`libjniOSX.dylib`) were compiled for Intel x86 chips, not Apple Silicon (M1/M2/M3), so your Mac cannot load them natively.

How to Fix It

Because Python Mode is currently unmaintained and hasn't been updated to support Processing 4's underlying Java architecture, you have two primary options depending on your preference.

Option 1: Downgrade to Processing 3 (Recommended for Python Mode)

If you want to keep using the Processing IDE with the exact

`add_library('sound')` syntax, the easiest path is to downgrade Processing.

1. Download **Processing 3.5.4** from the [Processing older releases page](#).
2. Open it, install **Python Mode** via the Contribution Manager, and try your sketch again.
3. Note: Because Processing 3.5.4 runs via Rosetta 2 on Apple Silicon, the Intel `.dylib` files will load correctly, and the older Java environment will support `add_library()`.

Option 2: Use p5 for Python (Modern Alternative)

If you prefer to stay on modern Python and take full advantage of your Apple Silicon chip without relying on an outdated Java wrapper, consider switching to the native Python implementation of Processing called **p5**.

Instead of running inside the Processing IDE, you write code in standard Python (using VS Code, Thonny, or any editor) and install it via your terminal:

Bash

```
pip install p5
```

A basic audio sketch looks similar to this:

Python

```
from p5 import *

# load the sound file
sound = Sound("sample.wav")
```

```
def setup():
    size(400, 400)
    sound.play()

def draw():
    background(220)

if __name__ == '__main__':
    run()
```

Option 3: Use the Java Mode with Python syntax (Python Mode RE)

There is a newer, community-maintained alternative mode inside Processing 4 called **Python Mode RE** (or using the **Python bindings for Processing 4** via the Thonny IDE plugin).

1. Open Processing 4.
2. Go to the Mode dropdown menu (top right) -> **Add Mode...**
3. Look for **Python Mode RE** or **Python Configurable Mode** if available, which addresses some of Processing 4's compatibility issues.

If you are just looking to get your current sketch up and running with minimal code changes, grabbing **Processing 3.5.4** is going to be your quickest relief! it works

Awesome! Glad to hear you're back up and running.

Have fun with the sound design, and let me know if you run into any other weird quirks while building your sketch!

however, when the audio is playing a spinning beach play displays and I cannot interact with the sketch. I cannot even close it until the sound has finished playing. This happens because of a threading clash between Processing's visual loop and the Java Sound library when triggered from inside Python.

Essentially, when you call your sound to play, it is accidentally hijacking the **Main Thread** (the UI thread that handles window clicks, drawing, and dragging) instead of spawning its own background thread. While the audio is running, the

window stops communicating with macOS, giving you the infamous spinning beach ball.

Depending on how your code is structured, here are the two ways to fix it:

Fix 1: The Most Likely Culprit (Move `.play()` out of `setup()`)

If you have `sound.play()` sitting directly inside your `setup()` function, the sketch will often freeze until the sound file completely finishes loading and playing before it ever triggers the first frame of `draw()`.

Try moving the initialization to `setup()`, but don't play it until a key press, a mouse click, or a conditional state inside `draw()`:

Python

```
add_library('sound')

def setup():
    size(400, 400)
    global sample
    # Load the file here
    sample = SoundFile(this, "your_audio.wav")

def draw():
    background(50)
    # Your visual code here

def mousePressed():
    # Triggering it via an event keeps it from blocking setup
    global sample
    sample.play()
```

Fix 2: Force it onto a Python Background Thread

If the sound still freezes your window even when played via a mouse click, you can use Python's built-in `threading` module to force the sound to play on a

completely separate channel, leaving the Processing window free to run smoothly.

Update your sketch to look like this:

Python

```
import threading
add_library('sound')

def setup():
    size(400, 400)
    global sample
    sample = SoundFile(this, "your_audio.wav")

def draw():
    background(50)
    # The beach ball should be gone, allowing you to interact
    here

def play_audio_async():
    global sample
    sample.play()

def mousePressed():
    # Instead of calling sample.play() directly,
    # we spin it up on its own background thread:
    t = threading.Thread(target=play_audio_async)
    t.start()
```

Why this works:

By wrapping the playback in `threading.Thread()`, Python tells the operating system, "Hey, run this audio playback code on a side-channel, and keep the main Processing window fully active." Your beach ball should instantly disappear, and the close (X) button will work normally again.

will the threading work with

```
from p5 import *
```

from playsound import playsound # Import the independent sound module

Yes, the `threading` concept will absolutely work with Python's standalone `p5` library and an external sound library!

However, **do not use the `playsound` module.**

The original `playsound` package is ancient, abandoned, and fundamentally broken on modern versions of Python and Apple Silicon (it causes installation crashes and terminal errors). Instead, use the modern, community-maintained drop-in replacement called `playsound3`.

You can install it via your terminal:

Bash

```
pip install playsound3
```

Here is exactly how you would structure the code to ensure `p5` keeps rendering smoothly while your sound plays on a background thread:

Python

```
from p5 import *
from playsound3 import playsound # The updated, Apple
Silicon-compatible version
import threading

def setup():
    size(400, 400)
    # No audio loading required in setup, playsound handles
    it on the fly

def draw():
    background(240)
    fill(0)
    text("Click anywhere to play audio without freezing!",
    50, 200)
```

```
def play_audio_worker():
    # This runs entirely on a side thread
    # block=True forces the THREAD to wait for the sound to
    finish,
    # but it won't freeze your p5 window!
    playsound("your_audio.wav", block=True)

def mouse_pressed():
    # Spin up the audio on a separate thread whenever the
    mouse is clicked
    audio_thread = threading.Thread(target=play_audio_worker)
    audio_thread.start()

if __name__ == '__main__':
    run()
```

A Crucial Detail with `playsound3`

Notice the `block=True` argument inside `playsound()`.

When running inside a background thread, you actually want the sound function to block. This keeps the background thread alive until the track finishes playing. Meanwhile, the main thread handling your p5 graphics, loops, and mouse inputs remains completely unbothered.

Sound keeps playing for a while even though the sketch was closed

This is a classic side effect of using detached background threads.

When you click the "Close" button on a p5 window, you are killing the **main thread** (the graphics loop). However, because Python's default behavior is to keep running as long as any thread is alive, your audio worker thread keeps happily chugging along in the background until the sound file finishes playing.

To fix this, you need to make your background thread a **daemon thread**.

The Fix: Daemon Threads

A "daemon" thread is a background thread that automatically dies the exact millisecond the main program exits. If you close the p5 window, the daemon thread is instantly terminated by Python, cutting off the audio immediately. Here is how to update your code:

Python

```
from p5 import *
from playsound3 import playsound
import threading

def setup():
    size(400, 400)

def draw():
    background(240)

def play_audio_worker():
    playsound("your_audio.wav", block=True)

def mouse_pressed():
    audio_thread = threading.Thread(target=play_audio_worker)

    #
    -----
    # CRITICAL LINE: This tells Python to kill this thread
    # the moment the main p5 window is closed.
    audio_thread.daemon = True
    #
    -----

    audio_thread.start()

if __name__ == '__main__':
    run()
```

Why this works:

By adding `audio_thread.daemon = True` right before you call `.start()`, you are linking its lifecycle to the window. If the sketch is running, the audio plays. If you kill the sketch, Python completely shuts down, and the music stops instantly.